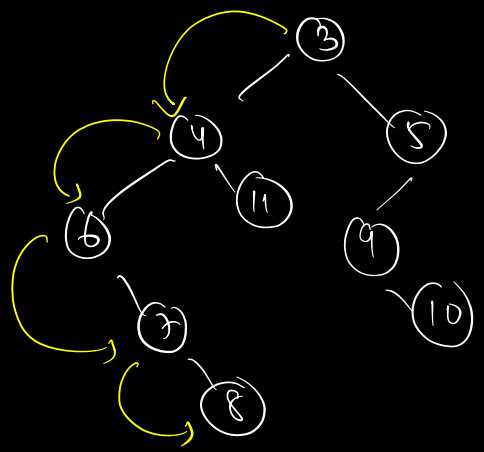
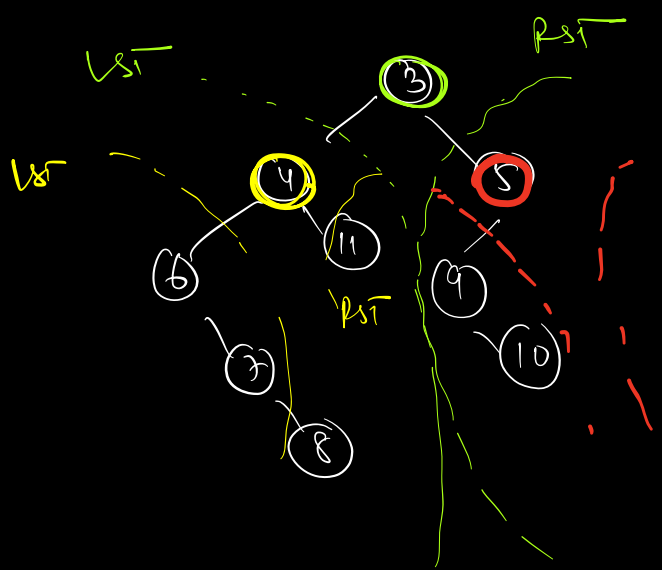
⇒ Height of a tree :-



h = 4 | 5



height(3) = 1 + max(height of LST, height of RST)

height(root) = [1 + max(height(root.left), height(root.right))]
return

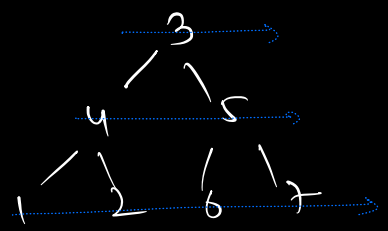
```
if(root == null)
    return 0
```

TC ⇒ O(N)
SC ⇒ O(h)
↳ depth of call stack

```
height( root ) {
    if( root == null )
        return 0;

    lh = height( root.left )
    rh = height( root.right )

    return max( lh, rh ) + 1
}
```



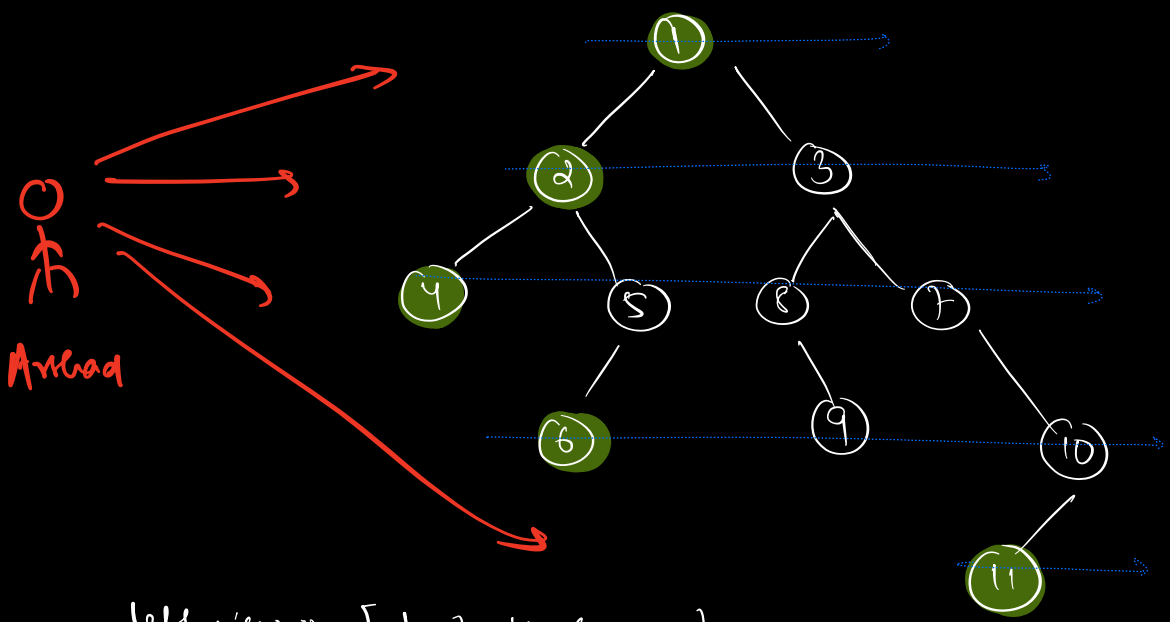
⇒ level order : 3 4 5 1 2 6 7
(L-R)

→ pushing element.left then element.right

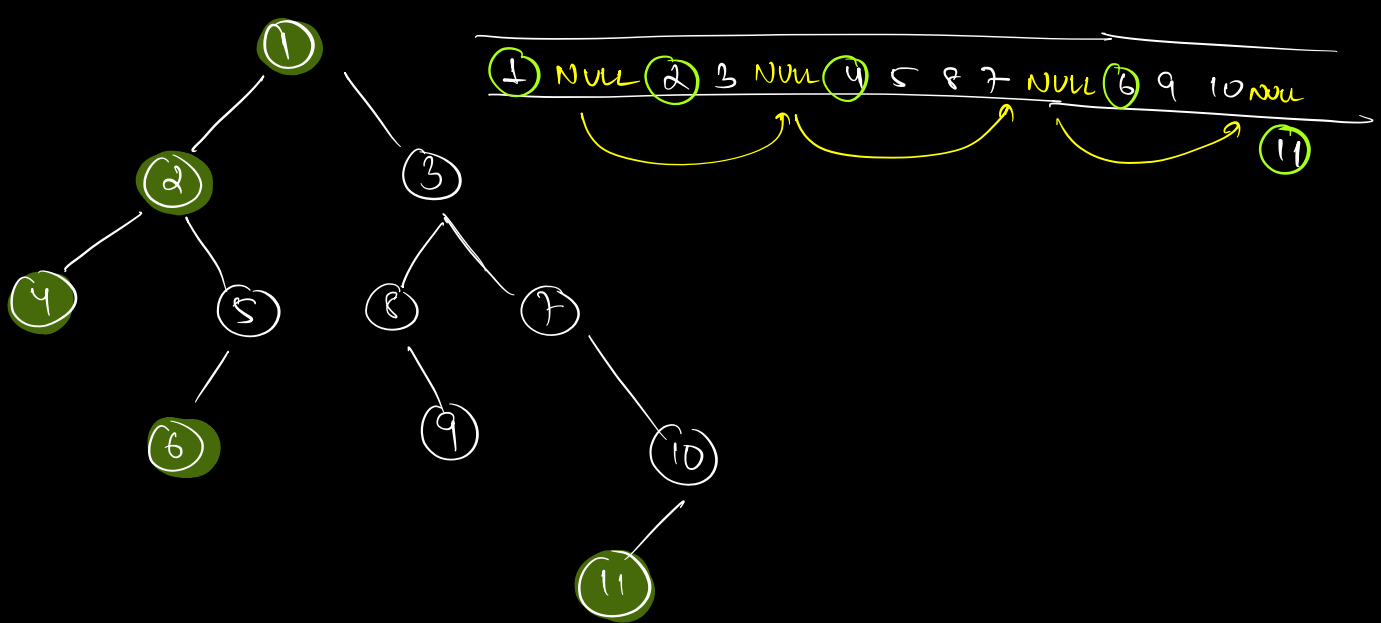
level order ⇒ 3 5 4 7 6 2 1
(R-L)

↳ push element.right then element.left

: left view of a tree:

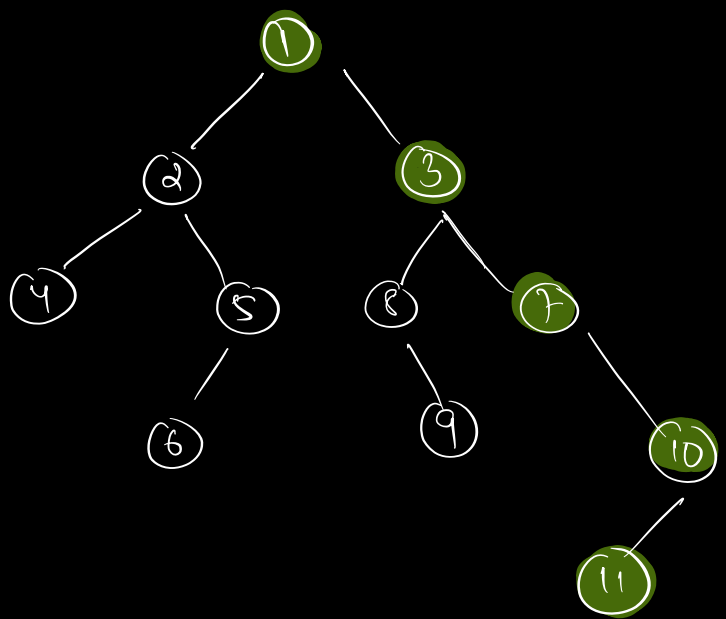


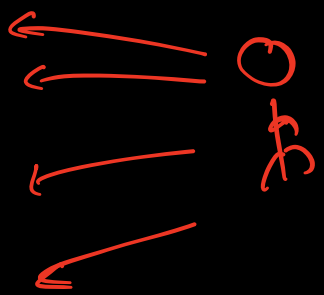
left view => [1, 2, 4, 6, 11]



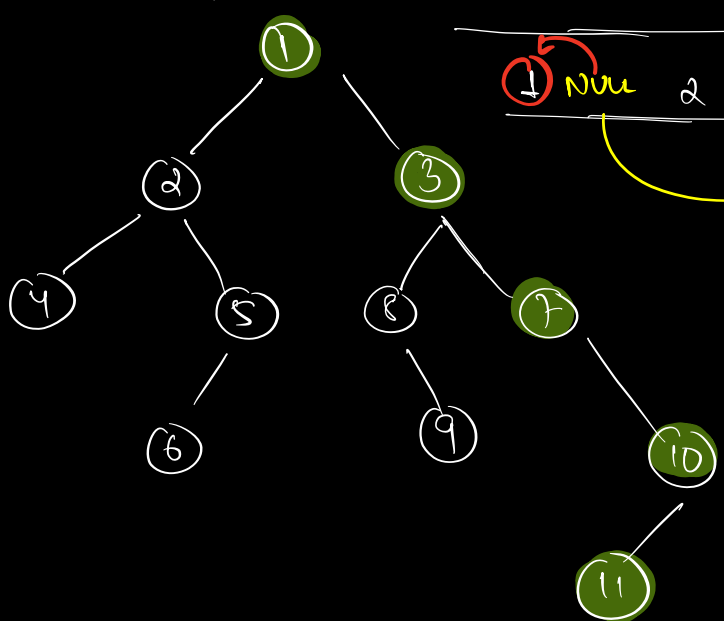
=> print root and element just after null in level wise level order traversal.

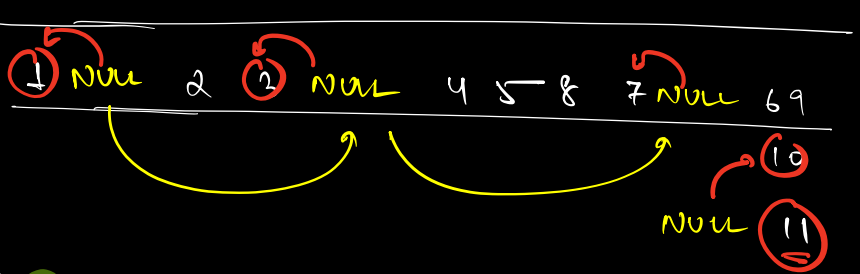
⇒ Right view of a tree :





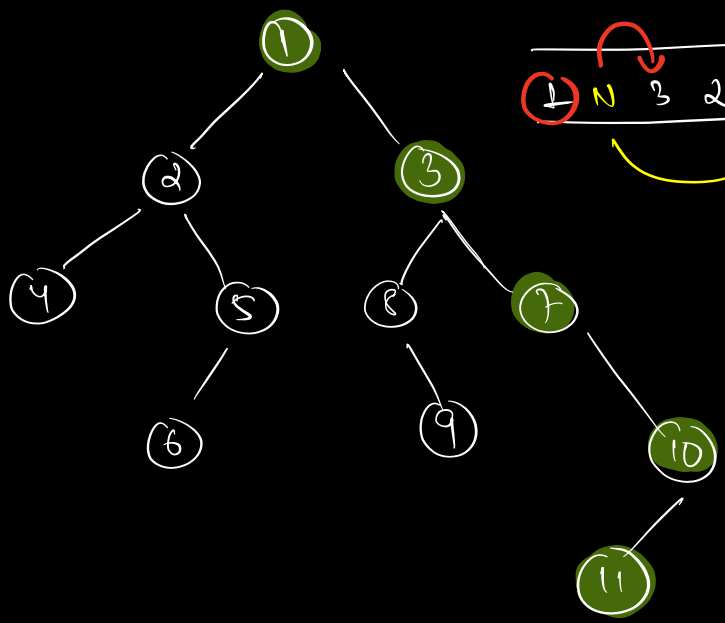
right view : [1, 3, 7, 10, 11]

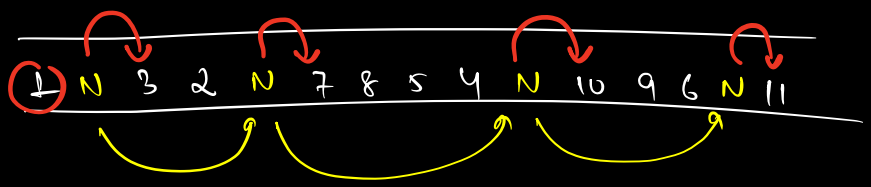




⇒ (L-R) level wise level order traversal

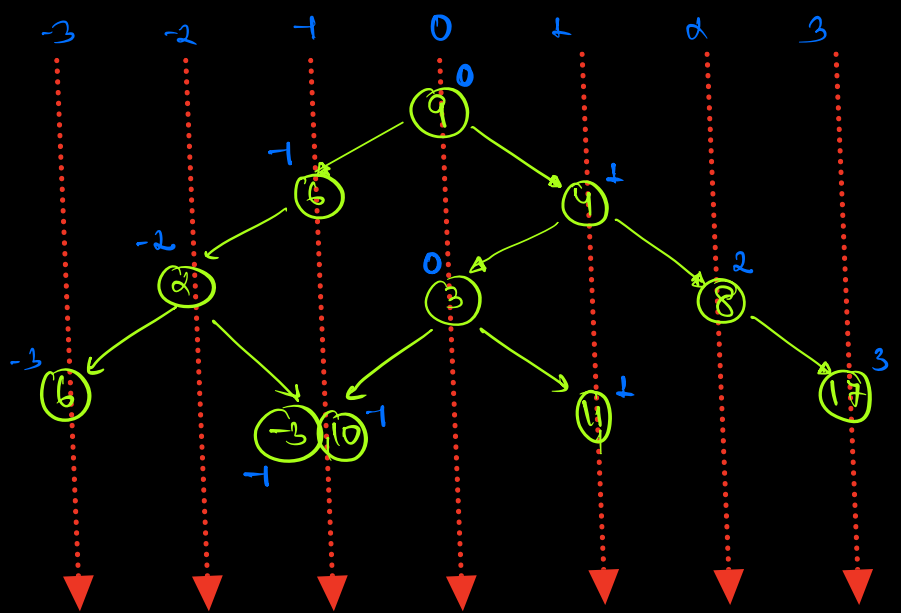
⇒ print element before NULL and last element





[⇒ (R-L) level wise level order traversal
⇒ print element just after NULL and root]

⇒ Vertical order traversal :-



print (L to R) ⇒ 6 2 6 -3 10 9 3 4 11 8 17
T to B

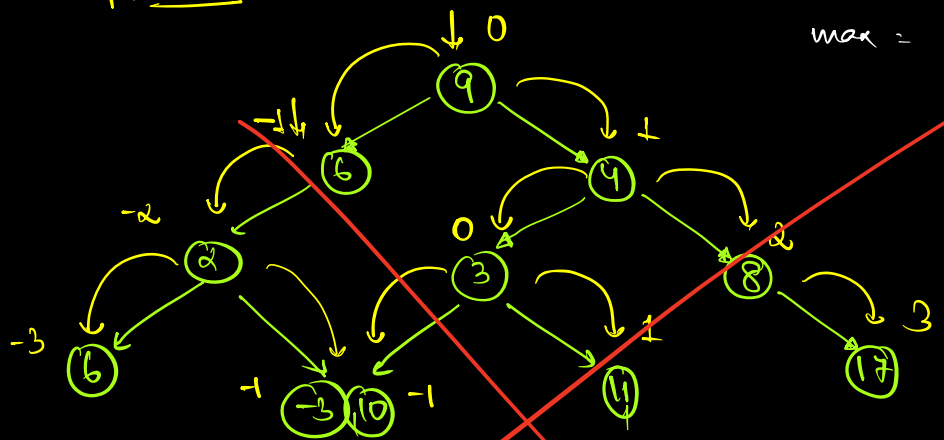
min(-3) to max(3)

min - max of axis (level)

⇒ Hashmap

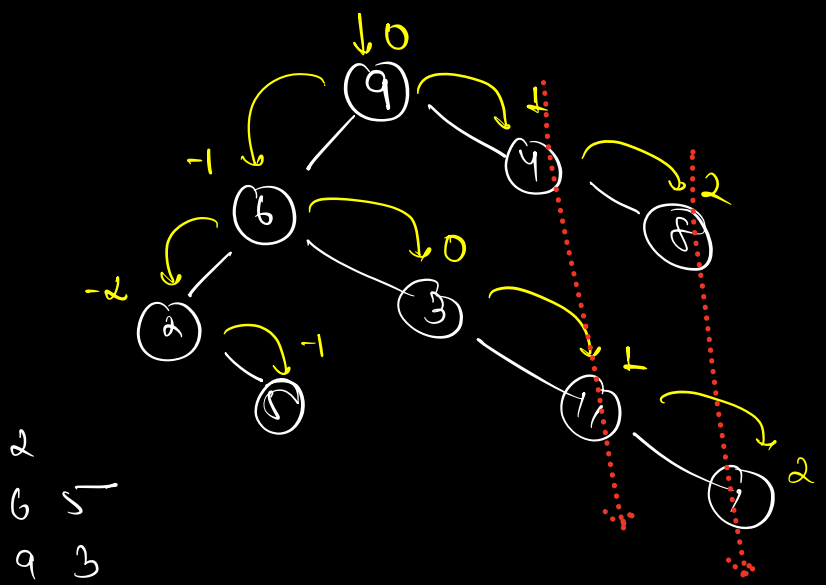
key	value
level	List<Nodes>
-2	2
-1	6 -3 10 → -3 10 6
2	8
3	7
0	9 3
1	4 11
-3	6

⇒ preorder



Map

0 → 9, 3
-1 → 6, -3, 10
-2 → 2
-3 → 6
1 → 4, 11
2 → 8
3 → 17

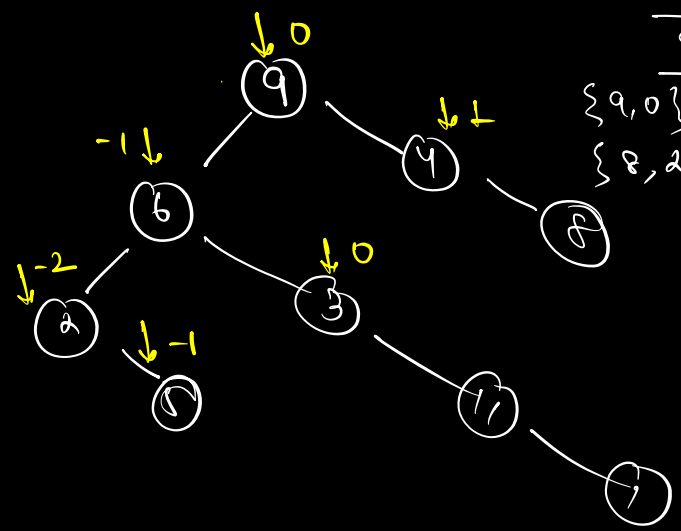


map

0 → 9, 3
-1 → 6, 5
-2 → 2
1 → 11, 4
2 → 1, 8

2
6 5
9 3
4 11
8 1

Class → { Node, level }



{1, 2}

{9,0} {6,-1} {4,1} {2,-2} {3,0}
{8,2} {5,-1} {11,1}

0 → 9, 3
-1 → 6, 5
1 → 4, 11
-2 → 2
2 → 8, 1

level

queue < Pair < Node, Int > > q

hashmap < level , list < Nodes > > map.

key value

q.enqueue ({ root, 0 });

while (!q.isEmpty()) {

x = q.dequeue();

update the hm(x)

minL = min (minL, x.level);

maxL = max (maxL, x.level);

if (x.Node.left != null) {

q.enqueue ({ x.Node.left, x.level - 1 })

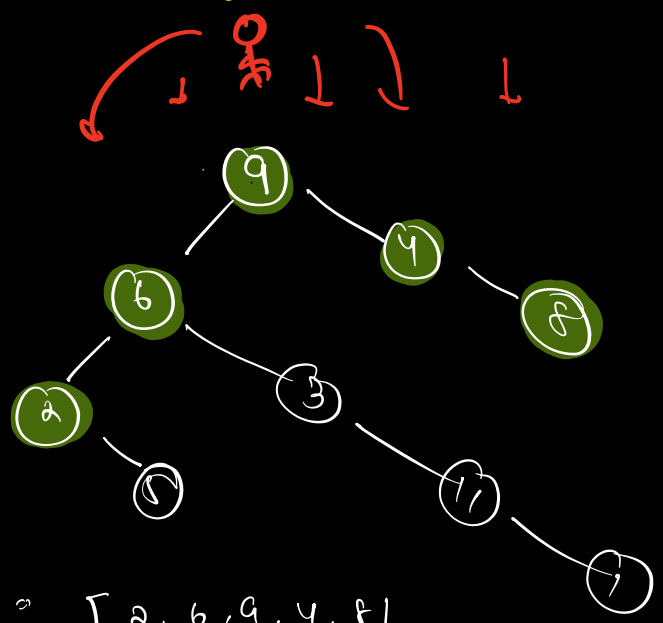
if (x.Node.right != null) {

q.enqueue ({ x.Node.right, x.level + 1 })

}

print HM from minL to maxL

⇒ Top view of a tree

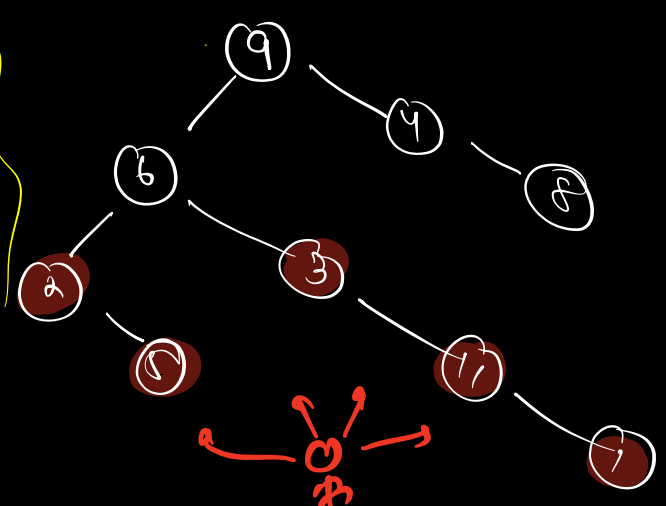


top view = [2, 6, 9, 4, 8]

[⇒ print 1st node for each level in vertical level traversal]

⇒ Bottom view of a tree:

[print last node for each level of hashmap]



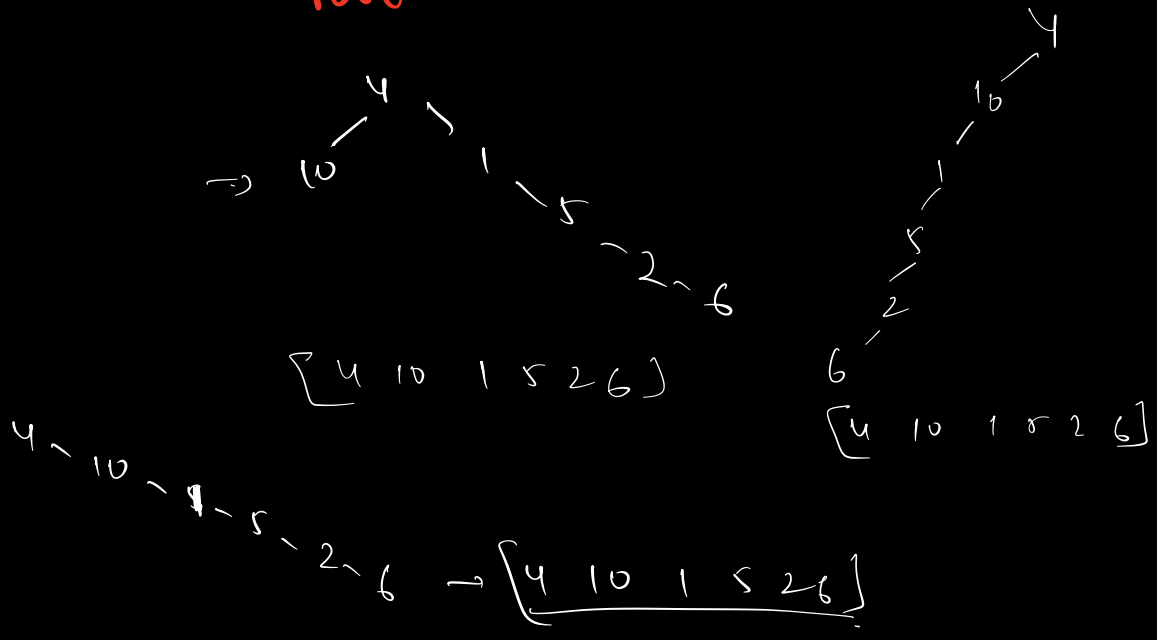
TC = O(N)
SC = O(N)

⇒ Construction of tree :

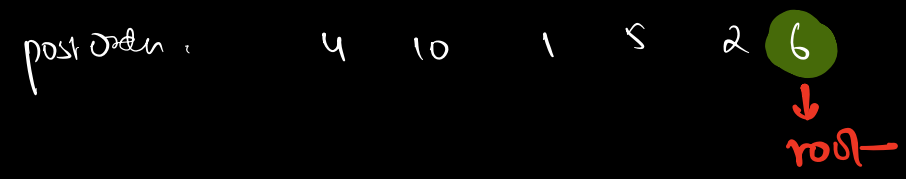
preorder ⇒ R l r

preorder → 4 10 1 5 2 6

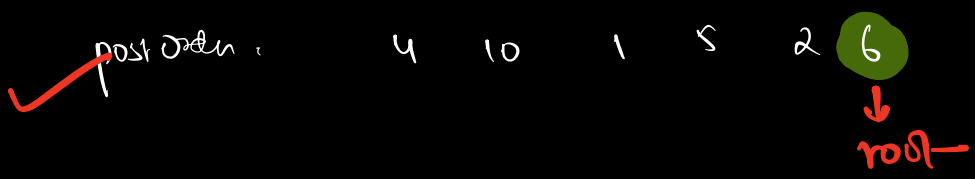
4 is root



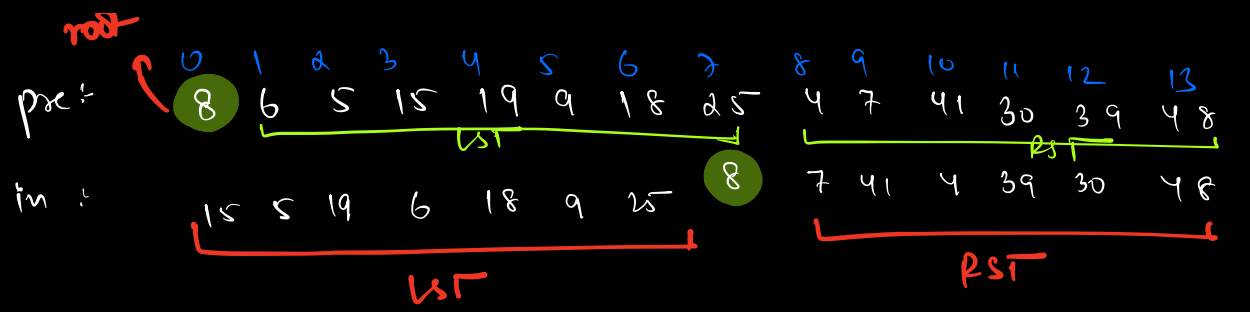
post order : l r ROOT

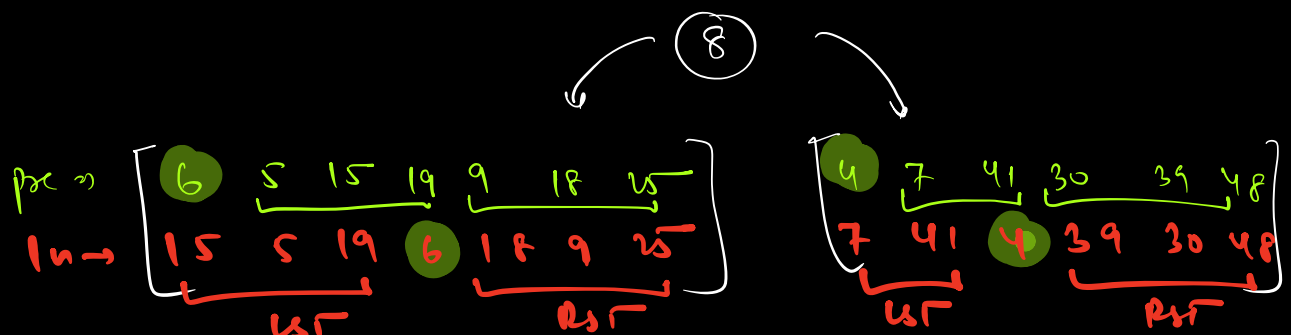


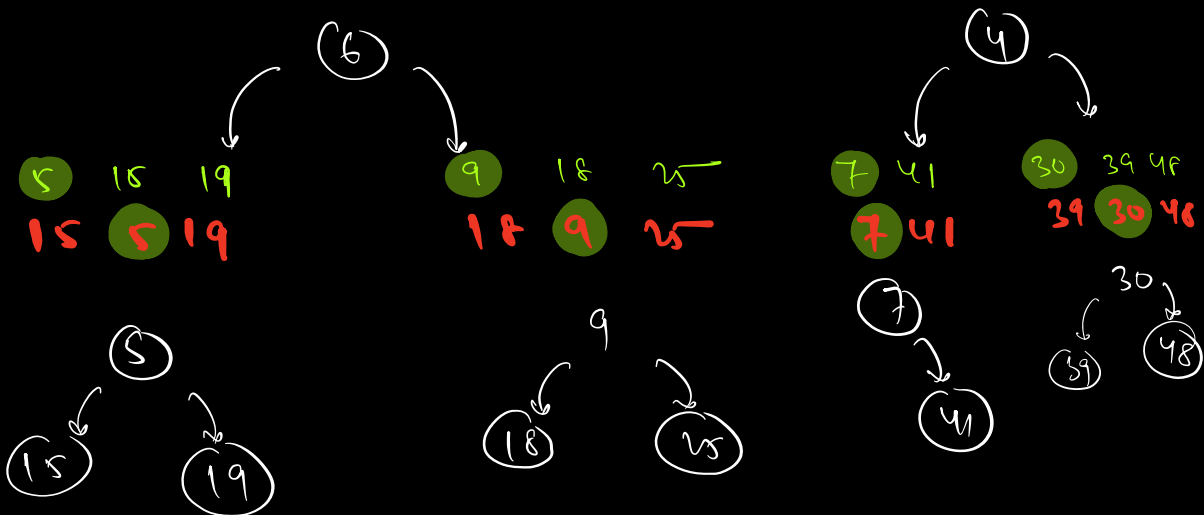
⇒ Inorder ⇒ l ROOT r

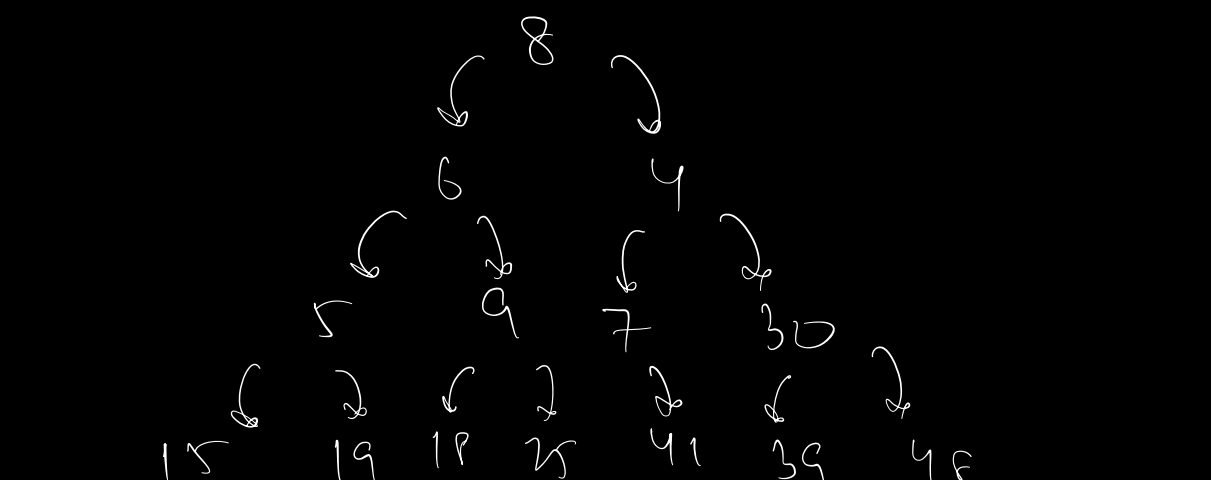


[Inorder + post order]
[Inorder + preorder]


root
pre :
0 1 2 3 4 5 6 7 8 9 10 11 12 13
8 6 5 15 19 9 18 25 4 7 41 30 39 48
LST
RST
8
in :
15 5 19 6 18 9 25 8 7 41 4 39 30 48
LST
RST

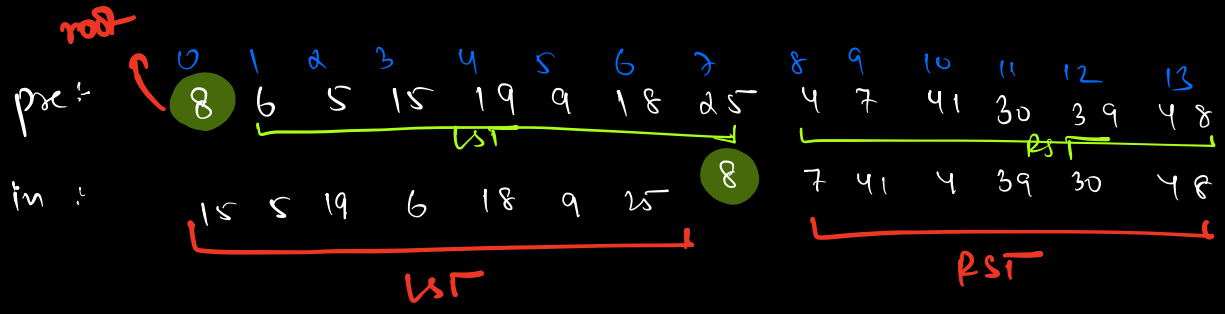

8
pre →
6 5 15 19 9 18 25 4 7 41 30 39 48
in →
15 5 19 6 18 9 25 7 41 4 39 30 48
LST
RST
LST
RST

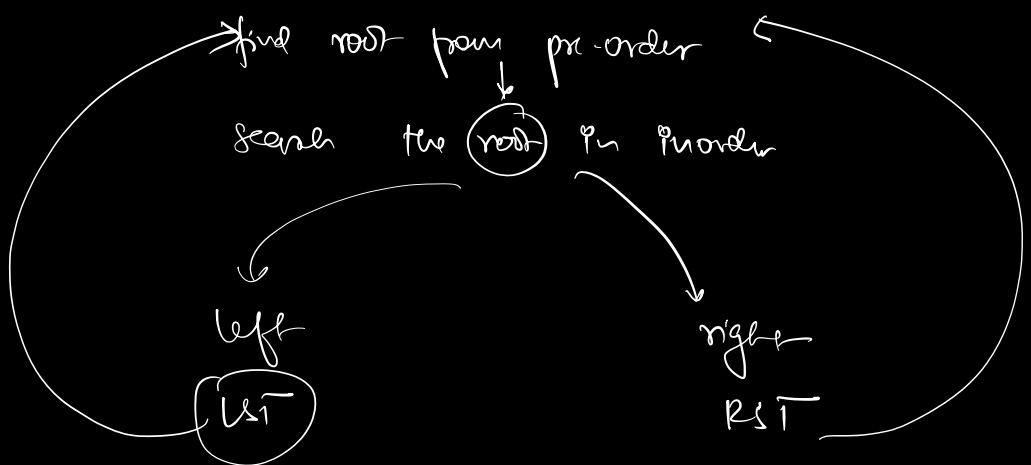

6
5 15 19
15 5 19
9 18 25
18 9 25
4
7 41
7 41
30 39 48
39 30 48
5
15
19
9
18
25
7
41
30
39
48


8
6
4
5
9
7
30
15
19
18
25
41
39
48

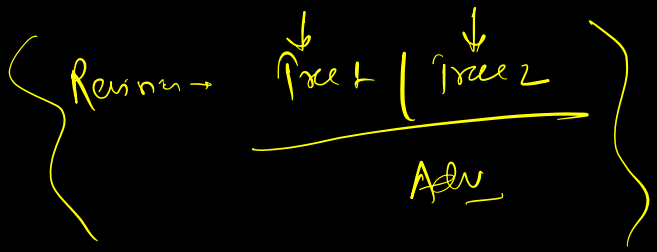
In → 15 5 19 6 18 9 25 8 7 41 4 39 30 48

Pre → 8 6 5 15 19 9 18 25 4 7 41 30 39 48





HW → pseudo code (HW)


Recursion
Tree 1
Tree 2
Adv